

Análisis de Requerimientos Funcionales

Actividad N°: 6 **Fecha:** 08/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Profundizar el backlog de requerimientos funcionales aprobado en la Semana 1, especificando reglas de negocio, entradas/salidas, prioridad y complejidad de cada uno, como insumo directo para el diseño técnico.

2. Especificación detallada de Requerimientos Funcionales

RF-01 — Autenticación de usuarios

- **Entrada:** usuario, contraseña.
- **Salida:** token de sesión (JWT) + datos del usuario (nombre, rol, punto de trabajo).
- **Regla de negocio:** un usuario inactivo (**activo: false**) no puede iniciar sesión aunque la contraseña sea correcta.
- **Prioridad:** Alta | **Complejidad:** Baja

RF-02 — Búsqueda de boletos

- **Entrada:** texto de búsqueda (nombre, apellido, email, cédula, Ticket ID o Transaction ID).
- **Salida:** listado paginado de boletos que coinciden.
- **Regla de negocio:** la búsqueda es insensible a mayúsculas/minúsculas; combina múltiples criterios con **\$and/\$or** según el caso.
- **Prioridad:** Alta | **Complejidad:** Media

RF-03 — Filtro por localidad y punto de trabajo

- **Entrada:** localidad (Seat), punto de trabajo.
- **Salida:** boletos filtrados.
- **Regla de negocio:** si el usuario autenticado es Staff, el filtro por punto de trabajo se aplica automáticamente y no puede ser cambiado por el usuario (se ignora cualquier valor enviado desde el cliente).
- **Prioridad:** Alta | **Complejidad:** Baja

RF-04 — Canje individual de boleto

- **Entrada:** Ticket ID, quién retira (Titular / Titular Compra / Otro), parentesco (si Otro), nombre de quien retira (si Otro), celular.
- **Salida:** boleto actualizado (**canjeado: true**), confirmación.
- **Reglas de negocio:**
 - **celular** es obligatorio siempre.
 - Si **quienRetira = "Otro"**, son obligatorios además **parentesco** y **quienOtro** (nombre).
 - Un boleto con **canjeado: true** no puede volver a canjearse (HTTP 400).

- **Prioridad:** Alta | **Complejidad:** Media

RF-05 — Bloqueo de doble canje

- **Regla de negocio:** validación a nivel de backend (no solo interfaz) antes de guardar el cambio, para evitar condiciones de carrera entre puntos de trabajo simultáneos.
- **Prioridad:** Alta | **Complejidad:** Media

RF-06 — Canje masivo

- **Entrada:** arreglo de Ticket IDs + un único formulario de datos de retiro.
- **Salida:** reporte de cuántos boletos se canjearon y cuántos ya estaban canjeados.
- **Regla de negocio:** los boletos ya canjeados dentro de la selección se excluyen automáticamente de la operación (no generan error, se informan aparte); la actualización se ejecuta como una sola operación `bulkWrite` para eficiencia.
- **Prioridad:** Media-Alta | **Complejidad:** Alta

RF-07 — Reimpresión controlada

- **Entrada:** Ticket ID, motivo.
- **Salida:** boleto con nueva entrada en su historial de reimpresiones.
- **Reglas de negocio:**
 - Solo permitido si el boleto ya fue canjeado/impreso previamente.
 - Motivo es obligatorio.
 - Solo accesible para rol Jefe.
- **Prioridad:** Media | **Complejidad:** Baja

RF-08 — Auditoría de operaciones

- **Entrada:** generada automáticamente por el sistema en cada operación relevante.
- **Salida:** registro de auditoría con tipo de operación, usuario, ticket, punto de trabajo, detalles e IP.
- **Regla de negocio:** un fallo al registrar auditoría **no debe** impedir que la operación principal (el canje) se complete; se registra el error de auditoría por separado.
- **Prioridad:** Alta | **Complejidad:** Baja

RF-09 — Gestión de usuarios

- **Entrada:** nombre, usuario, contraseña, rol, punto de trabajo (obligatorio si el rol es Staff).
- **Regla de negocio:** solo un usuario Jefe puede crear/editar/eliminar usuarios; un usuario Staff siempre debe tener un punto de trabajo asignado.
- **Prioridad:** Media | **Complejidad:** Baja

RF-10 — Gestión de puntos de venta y localidades

- **Entrada:** nombre del punto de venta, lista de localidades asociadas.
- **Regla de negocio:** las localidades disponibles para asociar provienen de los valores únicos de la columna "Seat" del CSV importado, no de una lista fija en el código.
- **Prioridad:** Alta | **Complejidad:** Media

RF-11 — Dashboard de estadísticas

- **Salida:** total de boletos, canjeados, pendientes, porcentaje de avance, evolución diaria, distribución por punto de trabajo.
- **Regla de negocio:** solo visible para rol Jefe; permite filtrar por punto de trabajo y rango de fechas.
- **Prioridad:** Media | **Complejidad:** Media

RF-12 — Importación desde CSV

- **Entrada:** archivo CSV con los datos de venta del evento.
- **Salida:** boletos cargados en la base de datos + punto de venta creado/actualizado con sus localidades.
- **Regla de negocio:** el **Ticket ID** es único; una re-importación no debe duplicar boletos existentes.
- **Prioridad:** Alta | **Complejidad:** Media

RF-13 — Actualización en tiempo real

- **Entrada:** eventos de canje/impresión generados por cualquier usuario conectado.
- **Salida:** evento **ticket-updated** emitido a las salas correspondientes (por punto de venta o punto de trabajo).
- **Regla de negocio:** la actualización no debe interrumpir al usuario si está escribiendo en un campo de búsqueda o interactuando con un formulario (pausa inteligente).
- **Prioridad:** Media | **Complejidad:** Alta

3. Priorización consolidada

Prioridad	Requerimientos
Alta	RF-01, RF-02, RF-03, RF-04, RF-05, RF-08, RF-10, RF-12
Media-Alta	RF-06
Media	RF-07, RF-09, RF-11, RF-13

4. Conclusiones del día

Cada requerimiento funcional cuenta ahora con reglas de negocio explícitas, lo cual permite pasar al análisis de requerimientos no funcionales y de seguridad sin ambigüedades sobre el comportamiento esperado del sistema.

Análisis de Requerimientos No Funcionales y Seguridad

Actividad N°: 7 **Fecha:** 09/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Definir en detalle los requerimientos no funcionales del sistema, con énfasis en seguridad, y establecer los mecanismos técnicos concretos que se usarán para cumplirlos.

2. Autenticación y autorización

Aspecto	Mecanismo definido
Autenticación	JSON Web Token (JWT), firmado con <code>JWT_SECRET</code> configurado por variable de entorno
Verificación de sesión	Middleware <code>auth</code> valida el token en cada request y recupera el usuario desde la base de datos, excluyendo el campo <code>password</code>
Usuarios inactivos	Un usuario con <code>activo: false</code> es rechazado aunque su token siga siendo técnicamente válido
Autorización por rol	Middleware <code>authorize(...roles)</code> bloquea el acceso a rutas según el rol del usuario autenticado (Jefe / Staff)
Contraseñas	Hasheadas con <code>bcryptjs</code> , salt de 12 rondas, nunca se devuelven en las respuestas JSON (<code>toJSON()</code> las elimina del objeto usuario)

3. Seguridad de la aplicación web

Mecanismo	Detalle
Helmet	Aplica cabeceras HTTP de seguridad por defecto (protección contra XSS, sniffing de MIME, clickjacking)
Rate limiting	Máximo 200 peticiones por minuto por IP; excluye rutas de <code>/health</code> y <code>check-changes</code> para no bloquear monitoreo ni sincronización legítima
CORS	Lista blanca explícita de orígenes permitidos en desarrollo (<code>localhost:3000</code> , <code>localhost:5173</code>) y en producción (dominios configurados + subdominios <code>*.onrender.com</code>); cualquier otro origen es rechazado
Validación de entrada	<code>express-validator</code> en el backend + validación de campos obligatorios en formularios del frontend
Manejo de errores	Middleware global que normaliza errores de validación, JWT y duplicados sin exponer detalles internos en producción

4. Seguridad de datos y auditoría

- Toda operación sensible (canje, impresión, reimpresión) genera un registro en **AuditLog** con usuario, IP y detalles — esto no es solo un requerimiento funcional, también es un control de seguridad (no repudio: un usuario no puede negar haber realizado una acción).
- Los datos de auditoría son de solo lectura desde la interfaz (no se exponen endpoints de edición/borrado de logs).
- El **Ticket ID** es único e indexado para impedir inconsistencias de datos entre reimportaciones.

5. Requerimientos No Funcionales (RNF) — versión final

ID	Categoría	Requerimiento
RNF-01	Seguridad	Autenticación JWT con expiración de sesión
RNF-02	Seguridad	Contraseñas con hash bcrypt (salt 12), nunca en texto plano ni en respuestas de API
RNF-03	Seguridad	Control de acceso por rol en cada endpoint sensible del backend
RNF-04	Seguridad	Cabeceras HTTP seguras (Helmet) y lista blanca de CORS
RNF-05	Seguridad	Límite de tasa de peticiones (rate limiting) para mitigar abuso/fuerza bruta
RNF-06	Disponibilidad	Soportar al menos 10 usuarios concurrentes sin degradación perceptible
RNF-07	Rendimiento	Búsquedas de boletos con tiempo de respuesta aceptable sobre miles de registros (índices en MongoDB: Ticket ID, Seat, updatedAt, compuestos)
RNF-08	Trazabilidad	Ninguna operación de canje/reimpresión debe perderse; auditoría inmutable de cada acción
RNF-09	Usabilidad	Interfaz operable con eficiencia bajo presión de tiempo (fila de personas esperando en el evento)
RNF-10	Portabilidad/Despliegue	Backend y frontend desplegables de forma independiente en la nube (Render), configurables por variables de entorno

6. Riesgos de seguridad identificados para mitigar en el diseño

- **Secretos en el código fuente:** se identifica como riesgo mantener credenciales de base de datos o secretos dentro de archivos versionados (ej. archivos de configuración de despliegue); deben manejarse siempre como variables de entorno gestionadas desde la plataforma de despliegue, nunca en texto plano en el repositorio.
- **Suplantación de punto de trabajo:** el filtro por punto de trabajo de un Staff debe forzarse en el backend (no confiar en el valor enviado desde el frontend).

- **Repetición de canje por condición de carrera:** dos solicitudes casi simultáneas para el mismo Ticket ID deben resolverse de forma que solo una tenga éxito.

7. Conclusiones del día

Se documentaron los mecanismos de seguridad ya implementados en el sistema (JWT, bcrypt, Helmet, rate limiting, CORS, RBAC, auditoría) y se formalizaron como requerimientos no funcionales, además de identificar riesgos a vigilar durante el desarrollo y despliegue.

Definición del Alcance del Sistema

Actividad N°: 8 **Fecha:** 10/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Delimitar formalmente qué incluye y qué NO incluye el sistema a desarrollar, evitando ambigüedades para las etapas de diseño y desarrollo.

2. Dentro del alcance (In Scope)

- Gestión y canje de boletos para **un evento a la vez** (caso real de referencia: concierto de Lumineers), con datos importados desde un archivo CSV entregado por la plataforma de venta.
- Búsqueda y filtrado de boletos por nombre, cédula, email, Ticket ID, Transaction ID y localidad (Seat).
- Canje individual y canje masivo de boletos, con registro de quién retira.
- Reimpresión controlada de boletos, restringida al rol Jefe.
- Gestión de usuarios internos del sistema (Jefe / Staff) y sus puntos de trabajo.
- Gestión de puntos de venta y localidades, extraídas dinámicamente del archivo importado.
- Auditoría completa de operaciones (canje, impresión, reimpresión, gestión de usuarios).
- Dashboard de estadísticas para el rol Jefe.
- Actualizaciones en tiempo real entre usuarios conectados (Socket.IO).
- Despliegue en la nube (backend y frontend) mediante Render.

3. Fuera del alcance (Out of Scope)

- **Venta de entradas en línea / pasarela de pagos:** el sistema no vende boletos ni procesa pagos; solo gestiona el canje de boletos ya vendidos por un canal externo.
- **Gestión de múltiples eventos/colecciones simultáneas:** en la versión actual el sistema trabaja sobre una única colección/evento activo a la vez; no se soporta operar varios eventos en paralelo desde la misma instancia.
- **Módulo de cronograma/Schedule:** existía en una versión anterior del proyecto y fue removido explícitamente del alcance por decisión de negocio (simplicidad operativa); no se reincorporará salvo nueva solicitud formal.
- **Aplicación móvil nativa:** el sistema se entrega como aplicación web responsiva, no como app nativa iOS/Android.
- **Facturación o control contable:** no se gestionan aspectos financieros/contables del evento, solo el control operativo del canje.
- **Integración directa con la plataforma de venta de entradas:** el ingreso de datos se realiza por importación de archivo CSV, no por integración API en tiempo real con el sistema de venta.

4. Supuestos

- El archivo CSV de venta se entrega completo y correcto antes del inicio del proceso de canje del evento.
- Cada boleto vendido tiene un **Ticket ID** único, que es la clave de identificación individual.

- La empresa dispone de conexión a internet estable en los puntos de canje el día del evento (requisito para tiempo real y autenticación contra el backend en la nube).

5. Restricciones

- Debe funcionar dentro de las capacidades del plan gratuito/estándar de MongoDB Atlas y Render usado por la empresa.
- El desarrollo debe mantenerse simple de operar para personal de staff sin conocimientos técnicos (usabilidad como restricción de diseño, no solo de UX).

6. Conclusiones del día

Con el alcance delimitado, quedan claros los límites del sistema frente a procesos que la empresa maneja por fuera (venta, pagos, facturación), lo que permite enfocar la planificación técnica exclusivamente en el proceso de canje.

Planificación Técnica de la Solución

Actividad N°: 9 **Fecha:** 11/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Definir la arquitectura tecnológica de la solución (stack, componentes, justificación de decisiones) y planificar técnicamente cómo se construirá el sistema durante las próximas semanas.

2. Stack tecnológico seleccionado

Backend

Tecnología	Uso
Node.js + Express	Servidor y API REST
MongoDB (Atlas) + Mongoose	Base de datos y modelado de esquemas
JSON Web Token (jsonwebtoken)	Autenticación de sesión
bcryptjs	Hash de contraseñas
Socket.IO	Comunicación en tiempo real
Helmet, express-rate-limit, cors	Seguridad de la API
Winston	Logging estructurado
express-validator	Validación de datos de entrada
csv-parser	Importación del archivo de venta

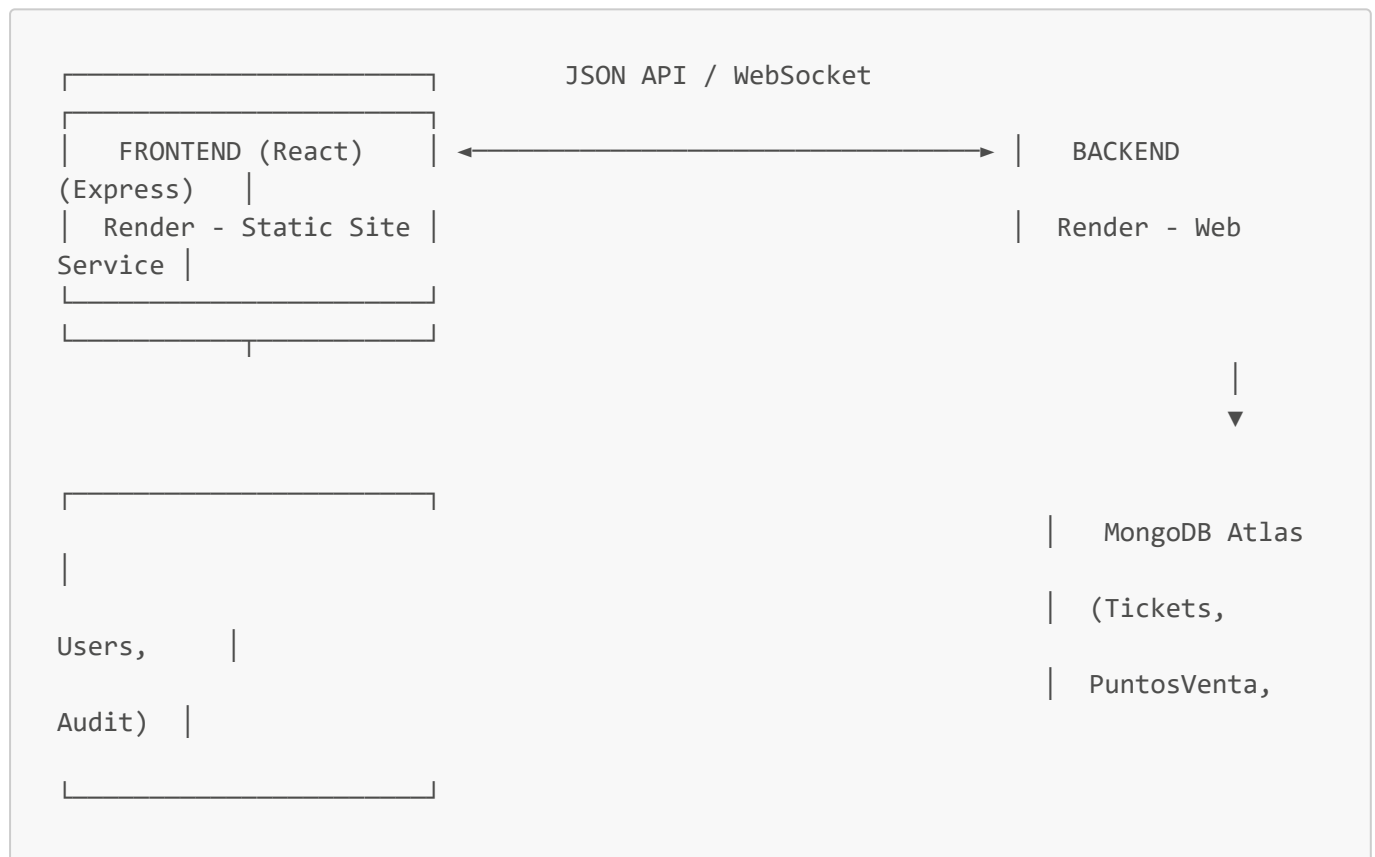
Frontend

Tecnología	Uso
React 18 + Vite	Interfaz de usuario y bundling
React Router v6	Enrutamiento entre páginas
React Bootstrap + Bootstrap 5	Componentes visuales
Context API	Estado global de autenticación
Axios	Consumo de la API REST
Socket.IO Client	Recepción de actualizaciones en tiempo real
Chart.js / react-chartjs-2	Gráficos del dashboard
SweetAlert2	Alertas y confirmaciones al usuario

3. Justificación de decisiones clave

- **MongoDB sobre SQL:** la estructura de un boleto varía poco pero el volumen y la necesidad de importar datos semi-estructurados de un CSV (columnas variables por evento) encaja mejor con un modelo de documentos flexible.
- **JWT sobre sesiones de servidor:** al desplegar backend y frontend como servicios independientes en Render (dominios distintos), un token portátil evita depender de cookies de sesión compartidas y simplifica el escalado horizontal.
- **Socket.IO sobre polling:** el requerimiento de "ver el cambio al instante entre varios puntos de venta" (evitar doble canje por desincronización) exige push en tiempo real; el polling periódico generaba carga innecesaria y latencia de varios segundos.
- **Arquitectura de colección única y localidades dinámicas:** al atender un evento a la vez con localidades que cambian de concierto en concierto, fijar las localidades en el código obligaría a un despliegue de código por cada evento; extraerlas del CSV en tiempo de importación elimina ese mantenimiento.

4. Arquitectura general de la solución



5. Plan de despliegue

- **Backend:** servicio web en Render, build `npm install`, start `npm start`, variables de entorno gestionadas desde el panel de Render (`MONGODB_URI`, `JWT_SECRET`, `CORS_ORIGIN`, etc.), nunca committeadas en texto plano en el repositorio.
- **Frontend:** sitio estático en Render, build `npm run build`, `VITE_API_URL` apuntando al backend desplegado; reglas de rewrite para servir la SPA (`index.html`) en todas las rutas.
- **Base de datos:** MongoDB Atlas, accesible por URI segura, con whitelist de IPs configurada para los servicios de Render.

6. Planificación de sprints (visión general)

Semana	Enfoque
3	Diseño de arquitectura, base de datos y componentes
4	Autenticación, gestión de usuarios y control de acceso
5	Funcionalidades de gestión, consulta, validación y canje
6	Control, seguimiento y consulta (auditoría, dashboard)
7	Integración y pruebas
8	Documentación y cierre

7. Conclusiones del día

Queda definida la arquitectura técnica completa de la solución y su justificación, lista para iniciar el diseño detallado en la Semana 3, así como el plan de despliegue en la nube.

Acta de Revisión y Aprobación de Requerimientos

Actividad N°: 10 **Fecha:** 12/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo de la reunión

Cerrar la etapa de análisis de requerimientos, presentando al tutor empresarial el paquete consolidado de la Semana 2 (requerimientos funcionales detallados, no funcionales y de seguridad, alcance y planificación técnica) para su aprobación formal antes de iniciar el diseño de la arquitectura.

2. Participantes

Nombre	Rol
Miguel Vivanco	Tutor Empresarial (FeelTheTickets)
Gabriel	Pasante / Desarrollador

3. Documentos presentados

- 06_Analisis_Requerimientos_Funcionales.md
- 07_Analisis_Requerimientos_No_Funcionales_Seguridad.md
- 08_Definicion_Alcance_Sistema.md
- 09_Planificacion_Tecnica_Solucion.md

4. Puntos revisados y resultado

Punto revisado	Resultado
Reglas de negocio de cada requerimiento funcional (RF-01 a RF-13)	✓ Aprobadas sin cambios
Priorización de requerimientos funcionales	✓ Aprobada
Requerimientos no funcionales y controles de seguridad (RNF-01 a RNF-10)	✓ Aprobados
Alcance del sistema (in scope / out of scope)	✓ Aprobado, se ratifica que venta, pagos y facturación quedan fuera del sistema
Stack tecnológico y arquitectura propuesta (MERN + Socket.IO + JWT)	✓ Aprobado
Plan de despliegue en Render y manejo de variables de entorno	✓ Aprobado, con énfasis en no exponer credenciales en el repositorio
Planificación de sprints (semanas 3 a 8)	✓ Aprobada como guía de trabajo

5. Observaciones del tutor empresarial

- Se solicita mantener especial atención al riesgo de doble canje en escenarios de alta concurrencia (varios puntos de venta activos al mismo tiempo), ya cubierto como RNF y regla de negocio de RF-05.
- Se ratifica que cualquier ampliación futura (por ejemplo, soportar más de un evento en simultáneo) se tratará como un requerimiento nuevo, fuera del alcance actual, y deberá pasar por el mismo proceso de validación.

6. Acuerdos y siguientes pasos

1. Se aprueba formalmente el cierre de la etapa de análisis de requerimientos.
 2. Se autoriza iniciar la Semana 3: diseño de arquitectura del sistema, base de datos y componentes de software.
 3. El backlog de requerimientos (funcionales y no funcionales) queda como línea base (*baseline*) para el resto del proyecto.
-